25.3-3
Show that if an edge (l, r) belongs to the directed equality subgraph $G_{M,h}$ but is not a member of $G_{M,h'}$, where h' is given by equation (25.5), then $l \in L - F_L$ and $r \in F_R$ at the time that h' is computed.

25.3-4
At line 29 in the FIND-AUGMENTING-PATH procedure, it has already been established that $v \in R$. This line checks to see whether v is already discovered by testing whether $v \in F_R$. Why doesn't the procedure need to check whether v is already discovered for the case when $v \in L$, in lines 26–28?

25.3-5
Professor Hrabosky asserts that the directed equality subgraph $G_{M,h}$ must be constructed and maintained by the Hungarian algorithm, so that line 6 of HUNGARIAN and line 15 of FIND-AUGMENTING-PATH are required. Argue that the professor is incorrect by showing how to determine whether an edge belongs to $E_{M,h}$ without explicitly constructing $G_{M,h}$.

25.3-6
How can you modify the Hungarian algorithm to find a matching of vertices in L to vertices in R that minimizes, rather than maximizes, the sum of the edge weights in the matching?

25.3-7
How can an assignment problem with $|L| \neq |R|$ be modified so that the Hungarian algorithm solves it?

Problems

25-1 Perfect matchings in a regular bipartite graph

a. Problem 20-3 asked about Euler tours in directed graphs. Prove that a connected, *undirected* graph $G = (V, E)$ has an Euler tour—a cycle

traversing each edge exactly once, though it may visit a vertex multiple times—if and only if the degree of every vertex in V is even.

b. Assuming that G is connected, undirected, and every vertex in V has even degree, give an $O(E)$-time algorithm to find an Euler tour of G, as in Problem 20-3(b).

c. Exercise 25.1-6 states that if $G = (V, E)$ is a d-regular bipartite graph, then it contains d disjoint perfect matchings. Suppose that d is an exact power of 2. Give an algorithm to find all d disjoint perfect matchings in a d-regular bipartite graph in $\Theta(E \lg d)$ time.

25-2 Reducing the running time of the Hungarian algorithm to $O(n^3)$

In this problem, you will show how to reduce the running time of the Hungarian algorithm from $O(n^4)$ to $O(n^3)$ by showing how to reduce the running time of the FIND-AUGMENTING-PATH procedure from $O(n^3)$ to $O(n^2)$. Exercise 25.3-5 demonstrates that line 6 of HUNGARIAN and line 15 of FIND-AUGMENTING-PATH are unnecessary. Now you will show how to reduce the running time of each execution of line 10 in FIND-AUGMENTING-PATH to $O(n)$.

For each vertex $r \in R - F_R$, define a new attribute $r.\sigma$ where

$$r.\sigma = \min \{l.h + r.h - w(l, r) : l \in F_L\}.$$

That is, $r.\sigma$ indicates how close r is to being adjacent to some vertex $l \in F_L$ in the directed equality subgraph $G_{m,h}$. Initially, before placing any vertices into F_L, set $r.\sigma$ to ∞ for all $r \in R$.

a. Show how to compute δ in line 10 in $O(n)$ time, based on the σ attribute.

b. Show how to update all the σ attributes in $O(n)$ time after δ has been computed.

c. Show that updating all the σ attributes when F_L changes takes $O(n^2)$ time per call of FIND-AUGMENTING-PATH.

d. Conclude that the HUNGARIAN procedure can be implemented to run in $O(n^3)$ time.

25-3 Other matching problems

The Hungarian algorithm finds a maximum-weight perfect matching in a complete bipartite graph. It is possible to use the Hungarian algorithm to solve problems in other graphs by modifying the input graph, running the Hungarian algorithm, and then possibly modifying the output. Show how to solve the following matching problems in this manner.

a. Give an algorithm to find a maximum-weight matching in a weighted bipartite graph that is not necessarily complete and with all edge weights positive.

b. Redo part (a), but with edge weights allowed to also be 0 or negative.

c. A ***cycle cover*** in a directed graph, not necessarily bipartite, is a set of edge-disjoint directed cycles such that each vertex lies on at most one cycle. Given nonnegative edge weights $w(u, v)$, let C be the set of edges in a cycle cover, and define $w(C) = \Sigma_{(u,v)\in C} w(u, v)$ to be the weight of the cycle cover. Give an algorithm to find a maximum-weight cycle cover.

25-4 Fractional matchings

It is possible to define a ***fractional matching***. Given a graph $G = (V, E)$, we define a fractional matching x as a function $x : E \to [0, 1]$ (real numbers between 0 and 1, inclusive) such that for every vertex $u \in V$, we have $\Sigma_{(u,v)\in E} x(u, v) \le 1$. The value of a fractional matching is $\Sigma_{(u, v)\in E} x(u, v)$. The definition of a fractional matching is identical to that of a matching, except that a matching has the additional constraint that $x(u, v) \in \{0, 1\}$ for all edges $(u, v) \in E$. Given a graph, we let M^* denote a maximum matching and x^* denote a fractional matching with maximum value.

a. Argue that, for any bipartite graph, we must have $\Sigma_{(u,\, v) \in E} x^*(u, v) \geq |M^*|$.

b. Prove that, for any bipartite graph, we must have $\Sigma_{(u,\, v) \in E} x^*(e) \leq |M^*|$. (*Hint:* Give an algorithm that converts a fractional matching with an integer value to a matching.) Conclude that the maximum value of a fractional matching in a bipartite graph is the same as the size of the maximum cardinality matching.

c. We can define a fractional matching in a weighted graph in the same manner: the value of the matching is now $\Sigma_{(u,\, v) \in E} w(u, v)\, x(u, v)$. Extend the results of the previous parts to show that in a weighted bipartite graph, the maximum value of a weighted fractional matching is equal to the value of a maximum weighted matching.

d. In a general graph, the analogous results do not necessarily hold. Give an example of a small graph that is not bipartite for which the fractional matching with maximum value is not a maximum matching.

25-5 Computing vertex labels

You are given a complete bipartite graph $G = (V, E)$ with edge weights $w(l, r)$ for all $(l, r) \in E$. You are also given a maximum-weight perfect matching M^* for G. You wish to compute a feasible vertex labeling h such that M^* is a perfect matching in the equality subgraph G_h. That is, you want to compute a labeling h of vertices such that

$$l.h + r.h \geq w(l, r) \quad \text{for all } l \in L \text{ and } r \in R\,, \tag{25.6}$$

$$l.h + r.h = w(l, r) \quad \text{for all } (l, r) \in M^*\,. \tag{25.7}$$

(Requirement (25.6) holds for all edges, and the stronger requirement (25.7) holds for all edges in M^*.) Give an algorithm to compute the feasible vertex labeling h, and prove that it is correct. (*Hint:* Use the similarity between conditions (25.6) and (25.7) and some of the properties of shortest paths proved in Chapter 22, in particular the triangle inequality (Lemma 22.10) and the convergence property (Lemma 22.14.))

Chapter notes

Matching algorithms have a long history and have been central to many breakthroughs in algorithm design and analysis. The book by Lovász and Plummer [306] is an excellent reference on matching problems, and the chapter on matching in the book by Ahuja, Magnanti and Orlin [10] also has extensive references.

The Hopcroft-Karp algorithm is by Hopcroft and Karp [224]. Madry [308] gave an $\tilde{O}(E^{10/7})$-time algorithm, which is asymptotically faster than Hopcroft-Karp for sparse graphs.

Corollary 25.4 is due to Berge [53], and it also holds in graphs that are not bipartite. Matching in general graphs requires more complicated algorithms. The first polynomial-time algorithm, running in $O(V^4)$ time, is due to Edmonds [130] (in a paper that also introduced the notion of a polynomial-time algorithm). Like the bipartite case, this algorithm also uses augmenting paths, although the algorithm for finding augmenting paths in general graphs is more involved than the one for bipartite graphs. Subsequently, several $O(\sqrt{V}E)$-time algorithms appeared, including ones by Gabow and Tarjan [168] as part of an algorithm for weighted matching and a simpler one by Gabow [164].

The Hungarian algorithm is described in the book by Bondy and Murty [67] and is based on work by Kuhn [273] and Munkres [337]. Kuhn adopted the name "Hungarian algorithm" because the algorithm derived from work by the Hungarian mathematicians D. Kőnig and J. Egervéry. The algorithm is an early example of a primal-dual algorithm. A faster algorithm that runs in $O(\sqrt{V}E \log(VW))$ time, where the edge weights are integers from 0 to W, was given by Gabow and Tarjan [167], and an algorithm with the same time bound for maximum-weight matching in general graphs was given by Duan, Pettie, and Su [127].

The stable-marriage problem was first defined and analyzed by Gale and Shapley [169]. The stable-marriage problem has numerous variants. The books by Gusfield and Irving [203], Knuth [266], and Manlove [313] serve as excellent sources for cataloging and solving them.

[1] The definition of a complete bipartite graph differs from the definition of complete graph given on page 1167 because in a bipartite graph, there are no edges between vertices in L and no edges between vertices in R.

[2] Although marriage norms are changing, it's traditional to view the stable-marriage problem through the lens of heterosexual marriage.

Part VII Selected Topics

Introduction

This part contains a selection of algorithmic topics that extend and complement earlier material in this book. Some chapters introduce new models of computation such as circuits or parallel computers. Others cover specialized domains such as matrices or number theory. The last two chapters discuss some of the known limitations to the design of efficient algorithms and introduce techniques for coping with those limitations.

Chapter 26 presents an algorithmic model for parallel computing based on task-parallel computing, and more specifically, fork-join parallelism. The chapter introduces the basics of the model, showing how to quantify parallelism in terms of the measures of work and span. It then investigates several interesting fork-join algorithms, including algorithms for matrix multiplication and merge sorting.

An algorithm that receives its input over time, rather than having the entire input available at the start, is called an "online" algorithm. Chapter 27 examines techniques used in online algorithms, starting with the "toy" problem of how long to wait for an elevator before taking the stairs. It then studies the "move-to-front" heuristic for maintaining a linked list and finishes with the online version of the caching problem we saw back in Section 15.4. The analyses of these online algorithms are remarkable in that they prove that these algorithms, which do not know their future inputs, perform within a constant factor of optimal algorithms that know the future inputs.

Chapter 28 studies efficient algorithms for operating on matrices. It presents two general methods—LU decomposition and LUP decomposition—for solving linear equations by Gaussian elimination in $O(n^3)$ time. It also shows that matrix inversion and matrix multiplication can be performed equally fast. The chapter concludes by showing how to compute a least-squares approximate solution when a set of linear equations has no exact solution.

Chapter 29 studies how to model problems as linear programs, where the goal is to maximize or minimize an objective, given limited resources and competing constraints. Linear programming arises in a variety of practical application areas. The chapter also addresses the concept of "duality" which, by establishing that a maximization problem and minimization problem have the same objective value, helps to show that solutions to each are optimal.

Chapter 30 studies operations on polynomials and shows how to use a well-known signal-processing technique—the fast Fourier transform (FFT)—to multiply two degree-n polynomials in $O(n \lg n)$ time. It also derives a parallel circuit to compute the FFT.

Chapter 31 presents number-theoretic algorithms. After reviewing elementary number theory, it presents Euclid's algorithm for computing greatest common divisors. Next, it studies algorithms for solving modular linear equations and for raising one number to a power modulo another number. Then, it explores an important application of number-theoretic algorithms: the RSA public-key cryptosystem. This cryptosystem can be used not only to encrypt messages so that an adversary cannot read them, but also to provide digital signatures. The chapter finishes with the Miller-Rabin randomized primality test, which enables finding large primes efficiently—an essential requirement for the RSA system.

Chapter 32 studies the problem of finding all occurrences of a given pattern string in a given text string, a problem that arises frequently in text-editing programs. After examining the naive approach, the chapter presents an elegant approach due to Rabin and Karp. Then, after showing an efficient solution based on finite automata, the chapter presents the Knuth-Morris-Pratt algorithm, which modifies the

automaton-based algorithm to save space by cleverly preprocessing the pattern. The chapter finishes by studying suffix arrays, which can not only find a pattern in a text string, but can do quite a bit more, such as finding the longest repeated substring in a text and finding the longest common substring appearing in two texts.

Chapter 33 examines three algorithms within the expansive field of machine learning. Machine-learning algorithms are designed to take in vast amounts of data, devise hypotheses about patterns in the data, and test these hypotheses. The chapter starts with k-means clustering, which groups data elements into k classes based on how similar they are to each other. It then shows how to use the technique of multiplicative weights to make predictions accurately based on a set of "experts" of varying quality. Perhaps surprisingly, even without knowing which experts are reliable and which are not, you can predict almost as accurately as the most reliable expert. The chapter finishes with gradient descent, an optimization technique that finds a local minimum value for a function. Gradient descent has many applications, including finding parameter settings for many machine-learning models.

Chapter 34 concerns NP-complete problems. Many interesting computational problems are NP-complete, but no polynomial-time algorithm is known for solving any of them. This chapter presents techniques for determining when a problem is NP-complete, using them to prove several classic problems NP-complete: determining whether a graph has a hamiltonian cycle (a cycle that includes every vertex), determining whether a boolean formula is satisfiable (whether there exists an assignment of boolean values to its variables that causes the formula to evaluate to TRUE), and determining whether a given set of numbers has a subset that adds up to a given target value. The chapter also proves that the famous traveling-salesperson problem (find a shortest route that starts and ends at the same location and visits each of a set of locations once) is NP-complete.

Chapter 35 shows how to find approximate solutions to NP-complete problems efficiently by using approximation algorithms. For some NP-complete problems, approximate solutions that are near optimal are quite easy to produce, but for others even the best approximation algorithms known work progressively more poorly as the problem size

increases. Then, there are some problems for which investing increasing amounts of computation time yields increasingly better approximate solutions. This chapter illustrates these possibilities with the vertex-cover problem (unweighted and weighted versions), an optimization version of 3-CNF satisfiability, the traveling-salesperson problem, the set-covering problem, and the subset-sum problem.